Implementing an Object-Constraint Extension Without VM Support

Tim Felgentreff

Software Architecture Group
Hasso-Plattner-Institut
Tim.Felgentreff@hpi.uni-potsdam.de

Constraints provide a useful technique for ensuring that desired properties hold in an application. As a result, they have been used in a wide range of applications, including graphical layout, simulation, scheduling, and problem-solving. We have previously proposed a design, Babelsberg, for a family of object-constraint languages that cleanly integrates constraints with the underlying language in a way that respects encapsulation and standard object-oriented programming techniques. However, our work on Babelsberg has relied on modifying the underlying virtual machine, but that is not an option for web-based applications, which have become increasingly prominent.

We thus present an approach to implementing Babelsberg without virtual machine support, along with an implementation as a JavaScript extension. We demonstrate the resulting language, Babelsberg/JS, on a number of applications and provide performance measurements. Programs without constraints in Babelsberg/JS run at the same speed as pure JavaScript versions, while programs that do have constraints can still be run efficiently. Our design and implementation also incorporate incremental resolving to support interaction, as well as a novel cooperating solvers architecture that allows multiple solvers to work together to solve more difficult problems.

1 Introduction

Constraints are relations among objects that should hold. This could be that all parts in an electrical circuit simulation obey the laws of physics, that the rows in a Sudoku include each digit from 0 to 9, or that a streamed video plays smoothly in the presence of changing CPU and network load. We also want to support interactive use of constraints, for example, continuously re-satisfying a set of layout constraints on screen widgets as they are moved with the mouse. In addition, it is useful to extend the constraint formalism to allow soft constraints as well as required ones, where the system should try to satisfy the soft constraints if possible, but it is not an error if they cannot be satisfied. For example, we might have a soft constraint for video quality that we are willing to relax if necessary, given the current network load, or a desired ideal spacing between two widgets that again can be relaxed if need be. In the work reported here, we want to support constraints in a clean way in an object-oriented language running in a lightweight, web-based programming environment.

Figure 1 shows a screenshot from our prototype system implemented in the Lively Kernel environment [12] that illustrates the kinds of capabilities we want. It shows a

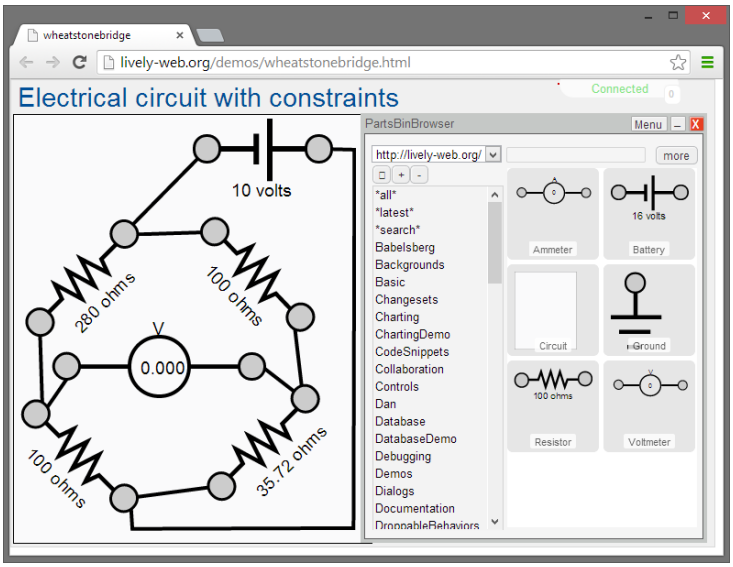


Figure 1: Constructing a Constraint-based Wheatstone Bridge Simulation

constraint-based simulation of Wheatstone Bridge being constructed. (A Wheatstone Bridge is used to measure an unknown electrical resistance by balancing two pairs of resistors so that the electrical potential between them is 0.) Parts representing batteries, resistors, and meters are copied from the Lively Kernel parts bin [12] on the right, dropped into the circuit on the left, and wired together. These parts carry constraints representing Ohm's Law, Kirchhoff's Current Law, and so forth. The system automatically solves the constraints when the parts are first connected, and re-solves them if the battery's supply voltage or a resistance is edited, updating the voltage displayed by the meter.

The capability to graphically construct constraint-based simulations dates back to Sketchpad [18] and ThingLab [1]. Babelsberg [6] enables a closer integration of constraints with the host object-oriented programming language in the spirit of constraint-imperative programming in Kaleidoscope [13] and Turtle [10].

We now want to support this design in the existing web-based environment. Our work is related to other approaches to constraint satisfaction in object-oriented programs, which use libraries [4, 19], domain-specific languages [5, 17], or (more recently) functional-reactive programming [14, 16] to specify and solve constraints. These approaches do not need special runtime support, but require the programmer to call specific application programming interfaces (APIs) or follow certain rules to not accidentally circumvent the constraints. With Babelsberg, we provide a syntax and semantics for unified declaration and execution of constraints and object-oriented program entities.

The first implementation of Babelsberg in Ruby, called Babelsberg/R [6], extended the Ruby Virtual Machine (VM). However, applications written in e.g. JavaScript typically have to work on a variety of client VMs included in different Web browsers. This makes it infeasible to implement Babelsberg in a JavaScript VM. JavaScript is currently of considerable interest in the industry and research communities. Thus, an implementation as an extension written entirely in JavaScript enables us to apply constraint programming to a variety of existing problems, and to compare it directly with alterna-

tive solutions on a variety of platforms.

A useful Object Constraint Programming language requires sufficiently powerful constraint solving capabilities. In prior work [6], we identified an important requirement, namely support for a new design for *cooperating constraint solvers* [2]. The motivation is that it is often infeasible to provide a single constraint solver that works well for all aspects of a problem; instead, different solvers may be more appropriate than others for some aspects, and which need to work together to solve the problem. Our design and implementation in Babelsberg/JS provides this capability, in a way that supports incremental re-solving of constraints without requiring access to the VM [7].

2 Object Constraint Programming Without VM Support

In industry, JavaScript has become the de-facto standard for Web programming, and a huge amount of code exists in the language. This fact, along with JavaScript's unique design and its execution environment in a Web browser, also make it of great interest to the research community, motivating work on revising and adapting useful features of other languages to include in it [11, 20].

To provide practical support for Object Constraint Programming (OCP) in JavaScript, we adapt the Babelsberg design to not require support from the underlying VM. This enables us to run Babelsberg/JS in modern browsers and use it in a variety of practical Web applications.

For Babelsberg/JS, since we do not have access to the VM, we cannot redefine the operation of LOAD and STORE instructions to handle variables with constraints on them. Instead, the unmodified JavaScript VM is used only for imperative evaluation mode. To intercept accesses and assignments to constrained variables, we wrap properties with property accessors that interact correctly with the constraint solver. To get the value of a constrained variable, the accessor gets the value for that variable from its solver. For a store, the setter in general calls the appropriate constraint solver to solve an equality constraint between the variable and its new value for a store.

For constraint construction mode, we use a custom JavaScript interpreter, itself written in JavaScript. This custom interpreter is about three orders of magnitude slower than the underlying one. However, since evaluating code in constraint construction mode is a much less common activity, and one that doesn't occur in inner loops, the performance penalty is not a significant issue.

Generalizing our approach, we have thus identified the following requirements for implementing the Babelsberg scheme without VM support:

- The host language must support a means to intercept variable lookup, so names can refer to different objects.
- The VM-based implementation ignores encapsulation and modify VM data structures directly. In contrast, here the extension must enable calling the appropriate API functions.
- The host language must provide a means to modify interpretation of a block of code to implement the constraint construction mode.

The first requirement is only partially supported in JavaScript, namely for object fields using property accessors. We therefore limit ourselves to constraining field storage in Babelsberg/JS, but not storage into local variables. (Some compiled OO languages, for example C#, also support property accessors; and other dynamic OO languages, such as Python and Smalltalk, support *method wrappers* to enable intercepting accessors, again within the limitation of only constraining field access.) As with the original Babelsberg/R design, it does not matter whether the fields are constrained directly or whether they are used in the execution of a method that was constrained to produce a certain result. A property that is accessed in the execution of a constraint expression is wrapped with property accessor that intercepts lookup and storage.

2.0.1 Property Accessors for Constrained Objects

When an object has been used in a constraint, its constrained properties have been replaced with property accessors. The *property getter* is a simple wrapper that reads from the solver variable in the most upstream region in which the field is referenced (cf. 2.1). Instead of returning the field value of the object, it returns the value of that variable in the solver data structure. The *property setter* distinguishes two cases. If the variable is writable from a solver, an equality constraint for that solver is created and the updated constraint system is solved, potentially triggering other solvers. On the other hand, if the variable is not writable (either because it is of a type that no available solver supports or because it has been marked as read-only by the programmer), its new value is stored, and all dependent constraints are recalculated. These dependent constraints have treated the variable as a constant (because they cannot modify it). To recalculate them, the constraints are deactivated in the solvers, and the expressions that created them are re-evaluated in constraint construction mode to create new constraints based on the new value. (The implementation of edit constraints (subsection 2.2) handles the situation of repeated changes much more efficiently.)

2.0.2 Creating Constraints

As an example of defining constraints, consider an interactive temperature converter, which maintains the relation between sliders representing values on the Fahrenheit, Celsius, Rankine, and Kelvin scales.

```
1 var converter = {},
2     cassowary = new CLSimplexSolver();
3 always: { solver:  cassowary
4     converter.C * 1.8 == converter.F - 32 &&
5     converter.C + 273.15 == converter.K &&
6     converter.F + 459.67 == converter.R
7 }
```

In Babelsberg/JS, a source-to-source transformation creates a call to a global function — **always** — from an **always**: expression of this form (this transformation just provides syntactic sugar – the function can also be called directly with function object.) Once this function has executed, a change to any one of the temperature values in the converter object will trigger changes to the other three values to keep the constraint satisfied through property accessors described above.

The **always** function passes the predicate expressing the constraint and information about the context into a custom JavaScript interpreter. This interpreter is used to eval-

uate expressions in constraint construction mode, which is provided as part of the Babelsberg/JS library. The custom interpreter creates property accessors (getters and setters) for the C, F, K, and R fields of the converter object. The appropriate accessor is then called whenever some other part of the program uses one of those fields. However, within the constraint expression, accesses to these fields do not use these accessors, but instead return *ConstrainedVariable* objects. Messages are then sent to these objects, and instead of calculating values, build up networks of primitive constraints that can then be satisfied by a solver. The **always** function returns a Constraint object that provides meta-level access to the asserted relations, using the protocol described for Babelsberg/R.

In this example, the constraints are on the fields of the object. However, constraints in Babelsberg/JS (as with any instance of the Babelsberg scheme) can also invoke methods that perform computations. For example, imagine the converter uses the getCelsius method to return a cached temperature value that is updated in regular intervals from a Web service:

```
var converter = {},
    cassowary = new CLSimplexSolver();

converter.getCelsius = function() {
    if (!converter.updater) {
        updateCelsius(converter); // updateCelsius omitted for brevity
        converter.updater = setInterval(5000, function() {
            updateCelsius(converter);
        });
    }
    return converter.C;
}

always: { solver:  cassowary
    converter.getCelsius() * 1.8 == converter.F - 32 &&
    converter.getCelsius() + 273.15 == converter.K &&
    converter.F + 459.67 == converter.R
}
```

By placing the constraint on the result of sending messages rather than on fields, Babelsberg respects object encapsulation. The value returned from the message send in this example is simply a float, but return values can also be arbitrary objects and computed values. For example, we could constrain the maximum pressure of a volume of dry air with a fixed density and gas constant, which would effectively limit the maximum temperature to around 36° Celsius.

```
converter.pressure = function () {
    var gasConstantDryAir = 287.058, // J/(kg * K)
        density = 1.293; // kg/m^3
    return density * gasConstantDryAir * converter.K / 1000;
}

always: { solver: cassowary
    converter.pressure() <= 115 // kPa
}
```

2.1 Cooperating Constraint Solvers

The temperature converter described above has no graphical representation. Cassowary only works on reals, yet in order to display the temperature scales, we need to convert the values into strings and update the Web browser's Document Object Model (DOM) using the appropriate API. This is best done with a local propagation solver, which can invoke arbitrary methods to satisfy the constraints, in this case by calling the API. (The constraints that define the temperature converter are simple enough that we could have used a local propagation solver for all of them, but this is unsatisfactory for many problems, such as the Wheatstone bridge example in Figure 1, since local propagation cannot handle such situations as simultaneous equations or inequalities.)

There is currently no single solver that can efficiently handle all constraints that arise in a typical application (and it seems unlikely that one can be created). To address this, we extend the work presented in [6] to include an architecture for cooperating constraint solvers, allowing a problem to be partitioned among multiple solvers. For this example, we use two solvers: one for linear arithmetic on the reals, and one for local propagation constraints.

Our architecture for cooperating solvers partitions constraints into regions that are connected via read-only variables, implementing the design proposed in [2]. The result is a very loose coupling among the cooperating solvers. This approach is in contrast to the more commonly-used Satisfiability Modulo Theory (SMT) technique for supporting cooperating constraint solvers [15], which uses inferred equality constraints as the means for the cooperating solvers to communicate (including the case when neither of the equated variables has a specific value). Our experience so far indicates that our approach is more suited to integration with imperative constructs, in which variables do always have specific values, and lends itself well to support edit constraints for incremental re-solving. (While we have not yet done so in our implementation, the architecture described in [2] in fact allows hierarchies of cooperating solvers, so that within a single region, there could be multiple solvers that cooperate by sharing inferred equality constraints.)

In the cooperating solvers architecture, each constraint belongs to exactly one solver. All constraints that belong to the same solver are in the same region. While constraints belong to exactly one region, variables may be shared across regions. This happens if variables occur in multiple constraints that belong to different regions. These variables must be read-only in all but one of the regions. Read-only variables are represented in a solver-specific manner, either using stay constraints for solvers that support them, or through required equality constraints. To support this, solver libraries should provide a method that makes a variable read-only for them.

In this architecture, the regions must form an acyclic graph, so that solving can simply proceed from the upstream to the downstream regions, propagating variable values. Figure 2 shows an example configuration. Solving proceeds from the left and each solver propagates values for its variables to downstream solvers that need them. The downstream solvers can only read, not write to those variables. This architecture prohibits loops and a system that oscillates without finding a solution. To create this graph, the system determines an order for the solvers based on the dependencies between the constraints. The programmer can explicitly control the position of a solver

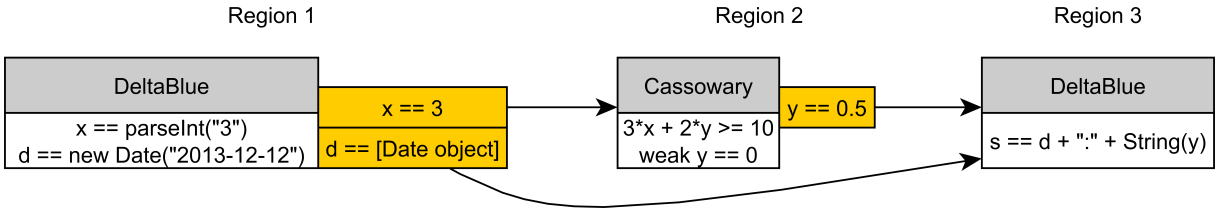


Figure 2: Regions propagating variable values downstream

in this graph, or the libraries can provide information so the system can create the order without the programmer's support. Applications can use multiple instances of the same solver type that are used one after the other (for example, for a problem that first uses Cassowary to solve simultaneous linear constraints, then DeltaBlue for local propagation constraints, then Cassowary again).

Once the solver regions are sorted, solving proceeds from the furthest upstream region. Each region will determine values for the variables it can write to, and the downstream regions will adjust to accommodate the new values propagated to their read-only variables from higher level regions. Soft constraints are solved for just within each region — in keeping with the theory of hard and soft constraints in the presence of read-only variables [3], if a soft constraint in an upstream region restricts a variable to a certain value, then a downstream region must use that value and can in fact not distinguish if this value was determined by a required or a soft constraint. If constraints in a downstream region cannot be satisfied due to an upstream soft constraint, we do not backtrack.

Given these additional capabilities, we can now add a graphical representation to our temperature converter. We want the color of a DIV element to change when the temperature is above 30° C.

```
1 var el = jQuery("#tooHotWarning");
2
3 always: { solver: deltablue
4     el.color.formula([converter.getCelsius()], function(celsius) {
5         var color = celsius > 30 ? "red" : "blue";
6         el.setAttribute("class", color);
7         return color;
8     });
9 }
```

Note that for the DeltaBlue local propagation solver, we do not provide a predicate (although we could — in that case it would be run to test whether re-solving is necessary). Instead, local propagation solvers need formulas for all writable variables that state their dependencies and how to update the variable. In this case, we want the Celsius value to be used as input for the color, but not vice versa, so we only provide one formula. The only dependency here is on the return value of converter.getCelsius(), passed explicitly in line 4. (Note that this could be omitted Babelsberg/R, because its version of DeltaBlue supports deducing the dependencies from the formula function — a feature we have not yet implemented here.) The dependencies are passed as arguments to the formula function, so we can use them directly to update the DOM using the browser's setAttribute API and return the new value. These functions, just like the predicates for Cassowary, are evaluated in constraint construction mode which wraps

variables with property accessors — the function formula is simply a function defined by DeltaBlue.

2.2 Incremental Re-Solving for Cooperating Constraint Solvers

Some applications involve repeatedly re-satisfying the same set of constraints with differing input values. A common such case is an interactive graphical application with a constrained figure, in which we move some part of the figure with the mouse. For such applications, it is important to re-solve the constraints efficiently, and a number of constraint solvers, including DeltaBlue and Cassowary, support this using edit constraints that allow a new value for a variable to be repeatedly input to the solver.

The original Babelsberg design did not include support for incremental re-solving at the language level — it was up to the solver library to provide access to such functionality. However, to integrate with our cooperating solvers architecture, Babelsberg/JS does include support for incremental re-solving through a solver-independent edit function that takes the variables to be edited and returns a callback function. The process that produces new values can use this callback to input new values into the solvers for the variables to be edited.

The edit function gathers all the constraints in which the passed variables participate. Only variables that occur solely in solver regions that support edit constraints can be edited; otherwise an exception is raised. The read-only annotations for variables in the solvers for downstream regions are converted to edit constraints, reflecting the fact that the upstream regions will be providing new values for these variables. Finally, the edit function creates a callback function and returns it. This callback can then be used to feed new values into the solvers.

As an example, suppose we wanted to connect the Celsius value of our temperature converter to a graphical slider. We wrap the original onDrag (which updates the slider's value) to input the new value into the edit callback as well.

```
1 var callback = edit(converter, ['C']);
2 slider.onDrag = slider.onDrag.wrap(function (originalOnDrag, evt) {
3     originalOnDrag(evt);
4     callback([slider.value]);
5 });
```

Two restrictions apply to the use of incremental re-solving with cooperating solvers: first, all variables that are edited must be only in regions of solvers that support edit constraints; and second, while the edit callback is used, no new constraints can be created. (Edit constraints are just a technique for optimizing the sequence of repeatedly replacing a constraint that a variable equal a constant with a new constraint with a new constant. Thus, if the restrictions aren't met, it is still possible to express and solve the desired constraints, just not as efficiently.)

3 Future Work and Conclusion

We have presented a design for implementing an Object Constraint Programming language without VM support, which is realized as a JavaScript extension called Babelsberg/JS.

We have also implemented a number of features from the original OCP design, including unified language constructs for constraint definition and object-oriented code, automatic maintenance of constraints, integration with the existing syntax and semantics, an interface to add new solvers and constraint solver constructs such as read-only variables and incremental re-solving; and also extended the design to support cooperating constraint solvers. There are a number of directions for future work.

Usability of Babelsberg/JS An important area for future work is the evaluation of the usability of our approach in general applications. We are interested in the comprehensibility of Babelsberg/JS code, especially to the target group for this language, i.e., imperative programmers with little prior experience with constraint programming. This will also provide opportunity to compare performance on more practical examples.

Debugging, Explanation, and Solver Selection It is currently difficult to tell why a solver may not be able to satisfy a given constraint, why it produced an unexpected result, or why finding a solution is slow. Our ConstraintInterpreter should include support for reasoning about the constraint system it builds. Prolog (or just a direct backtracking algorithm) may be useful as a "meta-solver" to automatically find a solver (or set of solvers) for a particular configuration of constraints. We are experimenting with a debugging tool for constraint construction in a Squeak/Smalltalk implementation of Babelsberg [9].

A Complete Formal Semantics The Babelsberg design has evolved alongside with its implementation. However, both because the implementations were driven in part by practical considerations about the expectations of programmers familiar with the underlying host language, as well as because of the complexities of integrating objects, state, and constraints, this has led to a number of semantic choices that are muddled with implementation specifics. For example, there have been a number of confusing issues with respect to constraints and object identity, how to represent assignment, what are the appropriate restrictions on expressions that define constraints, and what should happen if the solver can't find a solution to a set of constraints. In an effort to understand these better and to provide a complete design that instances of Babelsberg can implement, we are in the process of completing a formal operational semantics for Babelsberg [8].

⁂

Babelsberg/JS, compared to the earlier Babelsberg/R implementation, can be applied more directly to existing problems. It runs unmodified in different Web browsers, and integrates with the existing imperative language and libraries. The work reported here is quite recent, and we expect to continue to evolve both the language and its implementation.

References

[1] Alan Borning. The programming language aspects of ThingLab, a constraint-oriented simulation laboratory. *PLS*, 3(4):353–387, October 1981.

[2] Alan Borning. Architectures for cooperating constraint solvers. Technical Report VPRI Memo M-2012-003, Viewpoints Research Institute, Glendale, California, May 2012.

[3] Alan Borning, Bjorn Freeman-Benson, and Molly Wilson. Constraint hierarchies. *LISP and Symbolic Computation*, 5(3):223–270, 1992.

[4] Leonardo De Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *TACAS*, pages 337–340. Springer, March 2008.

[5] Enthought Inc. Enaml 0.6.3 documentation, February 2014.

[6] Tim Felgentreff, Alan Borning, and Robert Hirschfeld. Specifying and solving constraints on object behavior. *Journal of Object Technology*, 13(4):1:1–38, September 2014.

[7] Tim Felgentreff, Alan Borning, Robert Hirschfeld, Jens Lincke, Yoshiki Ohshima, Bert Freudenberg, and Robert Krahn. Babelsberg/JS. In *ECOOP*, pages 411–436. Springer, 2014.

[8] Tim Felgentreff, Todd Millstein, and Alan Borning. Developing a formal semantics for Babelsberg: A step-by-step approach. Technical Report 2014-002, Viewpoints Research Institute, September 2014.

[9] Maria Graber, Tim Felgentreff, Robert Hirschfeld, and Alan Borning. Solving interactive logic puzzles with object-constraints - an experience report using Babelsberg/S for Squeak/Smalltalk. In *REBLS*, pages 1:1–1:5. ACM, 2014. To appear.

[10] Martin Grabmüller and Petra Hofstedt. Turtle: A constraint imperative programming language. In *RDIS*, pages 185–198. Springer, 2004.

[11] Seonghoon Kang and Sukyoung Ryu. Formal specification of a JavaScript module system. In *OOPSLA*, pages 621–638. ACM, 2012.

[12] Jens Lincke, Robert Krahn, Dan Ingalls, Marko Roder, and Robert Hirschfeld. The Lively PartsBin–a cloud-based repository for collaborative development of active web content. In *HICSS*, pages 693–701. IEEE, January 2012.

[13] Gus Lopez, Bjorn Freeman-Benson, and Alan Borning. Kaleidoscope: A constraint imperative programming language. In *Constraint Programming*, volume 131, pages 313–329. Springer-Verlag, 1994. NATO Advanced Science Institute Series, Series F: Computer and System Sciences.

[14] Leo A Meyerovich, Arjun Guha, Jacob Baskin, Gregory H Cooper, Michael Greenberg, Aleks Bromfield, and Shriram Krishnamurthi. Flapjax: A programming language for Ajax applications. *SIGPLAN*, 44(10):1–20, 2009.

[15] Greg Nelson and Derek Oppen. Simplification by cooperating decision procedures. *PLS*, 1:245–257, 1979.

[16] Yoshiki Ohshima, Aran Lunzer, Bert Freudenberg, and Ted Kaehler. KScript and KSWorld: A time-aware and mostly declarative language and interactive GUI framework. In *Onward!*, pages 117–134, New York, 2013. ACM.

[17] Erica Sadun. *iOS Auto Layout Demystified*. Addison-Wesley, October 2013.

[18] Ivan Sutherland. Sketchpad: A man-machine graphical communication system. In *Proceedings of the Spring Joint Computer Conference*, pages 329–346. IFIPS, 1963.

[19] Emina Torlak and Daniel Jackson. Kodkod: A relational model finder. In *TACAS*, volume 4424, pages 632–647. Springer, April 2007.

[20] Tom Van Cutsem and Mark S Miller. Proxies: Design principles for robust object-oriented intercession APIs. *SIGPLAN*, 45(12):59–72, 2010.